Memoria

Arquitectura de computadores

Néstor Fuster,
Daniel Cholbi

VNIVERSITAT DE VALÈNCIA Escola Tècnica Superior d'Enginyeria

Índice

Explicación del código **2**
Manejo del modo de ejecución 2
Análisis de costes (FLOPs) **6**
Explicacion del metodo experimental **9**
Muestra Y Explicación de Resultados **11**
Código **12**

Explicación del código

Antes de explicar el código es importante recalcar que hemos decidido realizar la implementación del programa en el lenguaje C, puesto que este nos permite un control a mucho más bajo nivel que cualquier otro lenguaje, y para las optimizaciones que deseamos realizar en las próximas prácticas lo consideramos el más óptimo, pese a el aumento de dificultad a la hora de desarrollar el programa frente a C++.

El programa está estructurado en 6 bloques principales de ejecución secuencial:

Manejo del modo de ejecución

```
if (argc == 2 && strcmp(argv[1], "0") == 0) {
    debug_mode = 1;
    N = 6;
    D = 4;
} else if (argc == 3) {
    N = atoi(argv[1]);
    D = atoi(argv[2]);
    if (N <= 0 || D <= 0) {
        printf("Error: N y D deben ser enteros positivos.\n");
        return 1;
    }
} else {
    printf("Uso:\n");
    printf("  %s 0        (modo depuracion)\n", argv[0]);
    printf("  %s N D      (modo experimentacion)\n", argv[0]);
    return 1;
}
```

Lo primero es comprobar los argumentos que pasamos por la terminal. DSi introducimos un "0", el programa entra en modo depuración, utilizando tamaños fijos (N=6 y D=4) para que podamos comprobar que los cálculos son correctos. Si pasamos dos números, los usa como N y D para realizar los experimentos de rendimiento.

Reserva de memoria

```c
/* -------------------------
   2) Reserva de memoria
   Matrices en bloque contiguo
   ------------------------- */
double *X  = (double *) malloc(sizeof(double) * N * D);
double *C  = (double *) malloc(sizeof(double) * N * D);

double *WK = (double *) malloc(sizeof(double) * D * D);
double *WV = (double *) malloc(sizeof(double) * D * D);
double *WQ = (double *) malloc(sizeof(double) * D * D);

double *bK = (double *) malloc(sizeof(double) * D);
double *bV = (double *) malloc(sizeof(double) * D);
double *bQ = (double *) malloc(sizeof(double) * D);

double *K  = (double *) malloc(sizeof(double) * N * D);
double *V  = (double *) malloc(sizeof(double) * N * D);

double *q  = (double *) malloc(sizeof(double) * D); /* q_n temporal */
double *A  = (double *) malloc(sizeof(double) * N);
if (!X || !C || !WK || !WV || !WQ || !bK || !bV || !bQ || !K || !V || !q || !A) {
    printf("Error: no se pudo reservar memoria.\n");
    free(X); free(C);
    free(WK); free(WV); free(WQ);
    free(bK); free(bV); free(bQ);
    free(K); free(V);
    free(q); free(A);
    return 1;
}
```

Aquí utilizamos malloc para reservar el espacio de todas las matrices y vectores que necesita el algoritmo (X, C, pesos WK, WV, etc.). Un detalle importante es que creamos las matrices como un bloque de memoria contiguo de una sola dimensión. Para acceder a ellas como si fueran bidimensionales, simplemente aplicamos la fórmula de índices n*D+d.

Inicialización de datos

```
} else {
    /* Modo experimentación */
    srand((unsigned)time(NULL));

    /* X en [0, 10] */
    for (n = 0; n < N; n++) {
        for (d = 0; d < D; d++) {
            X[n * D + d] = 10.0 * ((double)rand() / (double)RAND_MAX);
        }
    }

    /* Pesos y sesgos pequeños en [-0.0005, 0.0005] */
    for (i = 0; i < D; i++) {
        for (j = 0; j < D; j++) {
            WK[i * D + j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
            WV[i * D + j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
            WQ[i * D + j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
        }
    }

    for (j = 0; j < D; j++) {
        bK[j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
        bV[j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
        bQ[j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
    }
}
```

- Los datos cambian dependiendo del modo en el que estemos ejecutando. En modo depuración, introducimos los valores fijos que marcaba el enunciado para poder verificar que el resultado final cuadra.
- En modo experimentación, se usa srand() para generar números pseudoaleatorios. Los pesos y sesgos se inicializan con valores aleatorios extremadamente pequeños (entre -0.0005 y 0.0005).

Algoritmo Self-Attention

Este es el bloque principal y el que realmente medimos con los tiempos t0 y t1. Se divide en dos fases:

- Primero, calculamos las matrices K y V de forma global para todo el programa. Esto consiste en multiplicar la entrada X por los pesos y sumar el sesgo.
- Segundo, mediante un bucle para cada iteración de n, calculamos el vector q. Luego sacamos la similitud multiplicando, dividimos por la raíz de D, aplicamos la función exponencial y normalizamos. Ese resultado se multiplica por las V que calculamos antes, obteniendo así el valor final en la matriz C

Salida y Liberación de memoria

```
/* -------------------------
   5) Salida
   ------------------------- */
if (debug_mode) {
    printf("C =\n");
    for (n = 0; n < N; n++) {
        for (d = 0; d < D; d++) {
            printf("%6.1f ", C[n * D + d]);
        }
        printf("\n");
    }
} else {
    printf("Tiempo: %f s\n", t1 - t0);
}

/* -------------------------
   6) Liberar memoria
   ------------------------- */
free(X);  free(C);
free(WK); free(WV); free(WQ);
free(bK); free(bV); free(bQ);
free(K);  free(V);
free(q);  free(A);
```

Para terminar, el programa imprime la matriz C si estamos en modo depuración, o muestra el tiempo de ejecución t1 - t0. Por último, utilizamos free() sobre cada puntero previamente reservado para evitar fugas de memoria y devolver los recursos al sistema.

Análisis de costes (FLOPs)

```
/* 4.1 Calcular K y V para todo n
   kn = WK^T * xn + bK
   vn = WV^T * xn + bV
*/
for (n = 0; n < N; n++) {
    for (j = 0; j < D; j++) {
        double sumK = bK[j];
        double sumV = bV[j];
        for (i = 0; i < D; i++) {
            int index_traspuesta = i * D + j;
            double xni = X[n * D + i];
            /* W^T * x  => usar W[i][j] */
            sumK += WK[index_traspuesta] * xni; // 2  FLOPS
            sumV += WV[index_traspuesta] * xni; // 2  FLOPS
        }
        int index = n * D + j;
        K[index] = sumK;
        V[index] = sumV;
    }
}
```

$$\sum_{n=0}^{N}\left(\sum_{j=0}^{D}\left(\sum_{i=0}^{D}(4)\right)\right) = N(4)$$

Todo lo que sigue a esta parte se encuentra dentro de un bucle de 0 a N por lo que todos los cálculos de costes de FLOPS están dentro de un sumatorio:

$$\sum_{n=0}^{N}()$$

```
/* 4.2 Para cada n: calcular q_n, similitudes, softmax y c_n */
for (n = 0; n < N; n++) {
    /* q_n = WQ^T * x_n + bQ */
    for (j = 0; j < D; j++) {
        double sumQ = bQ[j];
        for (i = 0; i < D; i++) {
            sumQ += WQ[i * D + j] * X[n * D + i]; // 2 FLOPS
        }
        q[j] = sumQ;
    }
```

$$\sum_{n=0}^{N}\left(\sum_{j=0}^{D}\left(\sum_{i=0}^{D}(2)\right)\right) = N(2D^2)$$

```
/* A(i) = exp( (q_n · k_i) / sqrt(D) ) */
double sum_exp = 0.0;
for (i = 0; i < N; i++) {
    double dot = 0.0;
    for (d = 0; d < D; d++) {
        dot += q[d] * K[i * D + d]; // 2 FLOPS
    }
    dot /= sqrtD; // 4 FLOPS
    A[i] = exp(dot); // 8 FLOPS
    sum_exp += A[i]; // 1 FLOPS
}
```

$$\sum_{n=0}^{N}\left(\sum_{j=0}^{D}\left(11 + \sum_{i=0}^{D}(4)\right)\right) = N(11D + 4D^2)$$

```
/* Normalizar A(i) */
for (i = 0; i < N; i++) {
    A[i] /= sum_exp; // 4 FLOPS
}
```

$$\sum_{n=0}^{N}\left(\sum_{j=0}^{D}\left(\sum_{i=0}^{D}(4)\right)\right) = N(4D)$$

```
/* c_n = sum_i A(i) * v_i */
for (d = 0; d < D; d++) {
    double sumC = 0.0;
    for (i = 0; i < N; i++) {
        sumC += A[i] * V[i * D + d]; // 2 FLOPS
    }
    C[n * D + d] = sumC;
}
```

$$\sum_{n=0}^{N}\left(\sum_{j=0}^{D}\left(\sum_{i=0}^{D}(2)\right)\right) = N(2D^2)$$

Calculadas cada parte del algoritmo solo queda sumar los costes, primero de todo sumaremos los últimos cálculos realizados:

$$N(2D^2) + N(4D) + N(11D + 4D^2) + N(2D^2) = N(15D + 8D^2)$$

Y para terminar los cálculos:

$$N(4D^2) + N(15D + 8D^2) = N(15D + 12D^2)$$

Entonces segun nuestros calculos los flops del grueso de nuestro programa varía como era de esperar en función de N y de D, llegando a tener un crecimiento cuadrático e incluso cúbico si es que N crece al mismo de D (Caso que consideraremos como tal en las ejecuciones)

Explicacion del metodo experimental

N	D	MFLOPS	No O2	O2	Mejora O2
100	100	12	0,011724	0,001519	7,7
200	200	97	0,090096	0,014356	6,3
300	300	325	0,276738	0,051864	5,3
400	400	770	0,609820	0,156923	3,9
500	500	1504	1,171503	0,277728	4,2
600	600	2597	2,003421	0,466840	4,3
700	700	4123	3,162355	0,721949	4,4
800	800	6154	4,707563	1,095857	4,3
900	900	8760	6,7059356	1,577340	4,3
1000	1000	12015	9,198798	2,146032	4,3
1100	1100	15990	12,234619	2,850729	4,3
1200	1200	20758	15,881744	3,538493	4,5
1300	1300	26389	20,215971	4,635960	4,4
1400	1400	32957	25,235538	5,713916	4,4
1500	1500	40534	31,347544	7,087544	4,4
1600	1600	49190	40,045233	14,248623	2,8
1700	1700	58999	46,489625	10,383834	4,5
1800	1800	70033	55,063363	12,443489	4,4
1900	1900	82362	67,457366	15,933959	4,2
2000	2000	96060	77,410754	17,715406	4,4
2100	2100	111198	92,578987	23,211767	4,0
2200	2200	127849	103,643719	25,686596	4,0
2300	2300	146083	127,385934	32,786341	3,9
2400	2400	165974	137,682313	37,262904	3,7
2500	2500	187594	166,682946	43,785477	3,8

A fin de poder observar correctamente el crecimiento del programa hemos decidido ejecutar el programa donde N = D, ya que si variamos N diferente de D los tiempos de testeo se dispararon especialmente en las simulaciones con mayores iteraciones.

Así pues, las muestras se realizarán empezando en N = 100 y D = 100 e iremos realizando incrementos de 100 hasta llegar a N, D = 2500.

Es importante recalcar que además el código será ejecutado inicialmente realizando una compilación normal, sin usar algoritmos de optimización tales como -O2, y posteriormente utilizaremos este último, compilando el programa con -O2 para ver la mejora en el rendimiento.

Por último, una vez realizadas las pruebas, mostraremos una tabla con los valores de N y D, los MFLOPS calculados con la fórmula calculada en el apartado anterior (Análisis de costes), los tiempos de ejecución con y sin la optimización de -O2 y por último una comparación de la mejoría en el tiempo de ejecución

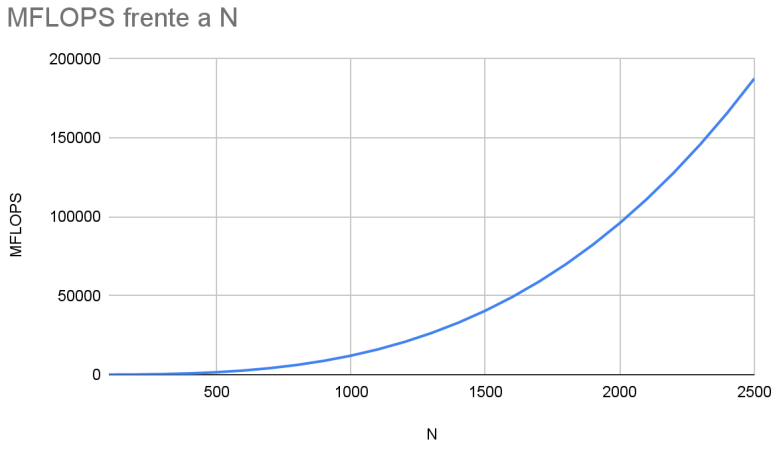
La máquina utilizada para las pruebas tiene las siguientes características:

OS: CachyOS x86_64
Kernel: Linux 6.19.3-2-cachyos

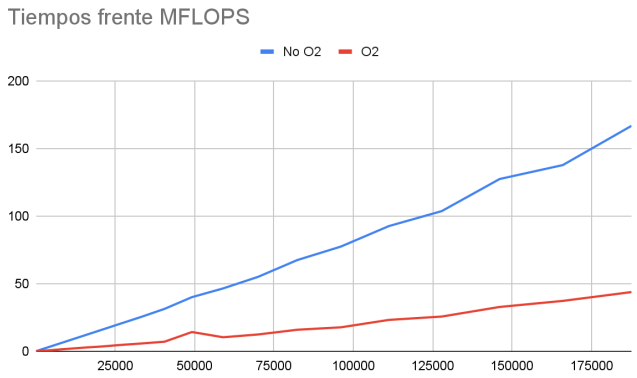
CPU: AMD Ryzen 7 9800X3D (16) @ 5.27 GHz
GPU 1: AMD Radeon RX 7900 XT [Discrete]
GPU 2: AMD Radeon Graphics [Integrated]
Memory: 30.51 GiB

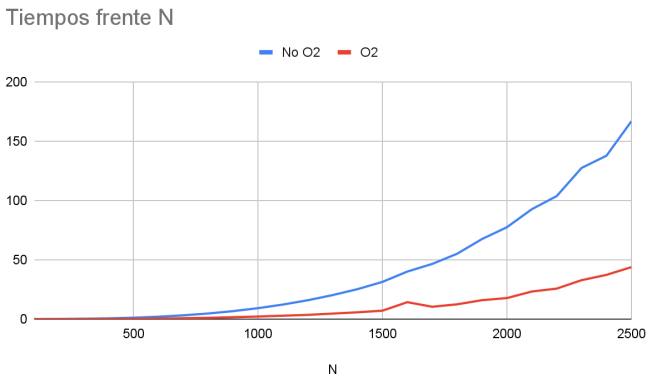
Aunque al ser un programa secuencial solo utilizará un procesador por lo que lo mas relevante es la frecuencia de dicho procesador.

Muestra Y Explicación de Resultados



Como se puede ver en la gráfica y como se explicó en su momento al calcular la fórmula, la cantidad de MFLOPS crece de forma cúbica a la talla del problema.





Aquí tenemos otras dos gráficas las cuales nos muestran el tiempo de ejecución tanto con -O2 tanto sin, marcados en rojo la ejecución usando -O2 y en azul sin. En la gráfica de la izquierda es más difícil ver puesto que se tiene que mirar comparándolo con la gráfica anterior, pero el tiempo de ejecución también crece de forma cúbica, al igual que los MFLOPS, y además, sin el uso de -O2 crece a un ritmo casi igual.

Adicionalmente se puede ver que la optimización de -O2 ha hecho un buen trabajo ya que pese a no hacer uso de paralelización, ha provocado que el programa se ejecute entre 7 a 4 veces más rápido, decreciendo la mejoría percibida a medida que se aumenta la talla del problema.

Código

```
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <time.h>

double get_time() {
   struct timespec t;
   clock_gettime(CLOCK_REALTIME, &t);
   return (double)t.tv_sec + ((double)t.tv_nsec) / 1e9;
}

int main(int argc, char *argv[]) {
   int debug_mode = 0;
   int N, D;
   int n, i, j, d;

   /* --------------------------
      1) Leer modo de ejecución
      -------------------------- */
   if (argc == 2 && strcmp(argv[1], "0") == 0) {
      debug_mode = 1;
      N = 6;
      D = 4;
   } else if (argc == 3) {
      N = atoi(argv[1]);
      D = atoi(argv[2]);
      if (N <= 0 || D <= 0) {
         printf("Error: N y D deben ser enteros positivos.\n");
         return 1;
      }
   } else {
      printf("Uso:\n");
      printf("  %s 0       (modo depuracion)\n", argv[0]);
      printf("  %s N D      (modo experimentacion)\n", argv[0]);
      return 1;
   }

   /* --------------------------
      2) Reserva de memoria
      Matrices en bloque contiguo
      -------------------------- */
   double *X  = (double *) malloc(sizeof(double) * N * D);
```

```
double *C  = (double *) malloc(sizeof(double) * N * D);

double *WK = (double *) malloc(sizeof(double) * D * D);
double *WV = (double *) malloc(sizeof(double) * D * D);
double *WQ = (double *) malloc(sizeof(double) * D * D);

double *bK = (double *) malloc(sizeof(double) * D);
double *bV = (double *) malloc(sizeof(double) * D);
double *bQ = (double *) malloc(sizeof(double) * D);

double *K  = (double *) malloc(sizeof(double) * N * D);
double *V  = (double *) malloc(sizeof(double) * N * D);

double *q  = (double *) malloc(sizeof(double) * D); /* q_n temporal */
double *A  = (double *) malloc(sizeof(double) * N); /* pesos atención temporales */

if (!X || !C || !WK || !WV || !WQ || !bK || !bV || !bQ || !K || !V || !q || !A) {
    printf("Error: no se pudo reservar memoria.\n");
    free(X); free(C);
    free(WK); free(WV); free(WQ);
    free(bK); free(bV); free(bQ);
    free(K); free(V);
    free(q); free(A);
    return 1;
}

/* -------------------------
   3) Inicialización de datos
   ------------------------- */
if (debug_mode) {
    /* X(n,d) según el enunciado:
       filas:
       0 6 12 18
       1 7 13 19
       ...
       5 11 17 23
       -> X[n*D + d] = n + N*d (con N=6)
    */
    for (n = 0; n < N; n++) {
        for (d = 0; d < D; d++) {
            X[n * D + d] = (double)(n + N * d);
        }
    }

    /* WK: todas las filas iguales a [-0.2, -0.1, 0.0, 0.1] */
    for (i = 0; i < D; i++) {
```

```
        for (j = 0; j < D; j++) {
            WK[i * D + j] = -0.2 + 0.1 * j;
        }
    }
    for (j = 0; j < D; j++) bK[j] = -1.0;

    /* WQ:
       fila 0: -0.2 -0.2 -0.2 -0.2
       fila 1: -0.1 -0.1 -0.1 -0.1
       fila 2:  0.0  0.0  0.0  0.0
       fila 3:  0.1  0.1  0.1  0.1
    */
    for (i = 0; i < D; i++) {
        double val = -0.2 + 0.1 * i;
        for (j = 0; j < D; j++) {
            WQ[i * D + j] = val;
        }
    }
    for (j = 0; j < D; j++) bQ[j] = 0.1;

    /* WV = identidad, bV = 0 */
    for (i = 0; i < D; i++) {
        for (j = 0; j < D; j++) {
            WV[i * D + j] = (i == j) ? 1.0 : 0.0;
        }
    }
    for (j = 0; j < D; j++) bV[j] = 0.0;

} else {
    /* Modo experimentación */
    srand((unsigned)time(NULL));

    /* X en [0, 10] */
    for (n = 0; n < N; n++) {
        for (d = 0; d < D; d++) {
            X[n * D + d] = 10.0 * ((double)rand() / (double)RAND_MAX);
        }
    }

    /* Pesos y sesgos pequeños en [-0.0005, 0.0005] */
    for (i = 0; i < D; i++) {
        for (j = 0; j < D; j++) {
            WK[i * D + j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
            WV[i * D + j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
            WQ[i * D + j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
        }
```

```
    }

    for (j = 0; j < D; j++) {
        bK[j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
        bV[j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
        bQ[j] = 0.001 * (((double)rand() / (double)RAND_MAX) - 0.5);
    }
}

/* --------------------------
   4) Algoritmo self-attention
   (misma implementación para ambos modos)
   -------------------------- */
double t0 = 0.0, t1 = 0.0;
if (!debug_mode) {
    t0 = get_time();
}

double sqrtD = sqrt((double)D);

/* 4.1 Calcular K y V para todo n
   kn = WK^T * xn + bK
   vn = WV^T * xn + bV
*/
for (n = 0; n < N; n++) {
    for (j = 0; j < D; j++) {
        double sumK = bK[j];
        double sumV = bV[j];
        for (i = 0; i < D; i++) {
            int index_traspuesta = i * D + j; // 2 FLOP
            double xni = X[n * D + i]; // 2 FLOP
            /* W^T * x  => usar W[i][j] */
            sumK += WK[index_traspuesta] * xni; // 2  FLOPS
            sumV += WV[index_traspuesta] * xni; // 2  FLOPS
        }
        int index = n * D + j; // 2 * D FLOP
        K[index] = sumK;
        V[index] = sumV;
    }
}

/* 4.2 Para cada n: calcular q_n, similitudes, softmax y c_n */
for (n = 0; n < N; n++) {
    /* q_n = WQ^T * x_n + bQ */
    for (j = 0; j < D; j++) {
        double sumQ = bQ[j];
```

```
        for (i = 0; i < D; i++) {
            sumQ += WQ[i * D + j] * X[n * D + i]; // 6 FLOPS
        }
        q[j] = sumQ;
    }

    /* A(i) = exp( (q_n · k_i) / sqrt(D) ) */
    double sum_exp = 0.0;
    for (i = 0; i < N; i++) {
        double dot = 0.0;
        for (d = 0; d < D; d++) {
            dot += q[d] * K[i * D + d]; // 4 FLOPS
        }
        dot /= sqrtD; // 4 FLOPS
        A[i] = exp(dot); // 8 FLOPS
        sum_exp += A[i]; // 1 FLOPS
    }

    /* Normalizar A(i) */
    for (i = 0; i < N; i++) {
        A[i] /= sum_exp; // 4 FLOPS
    }

    /* c_n = sum_i A(i) * v_i */
    for (d = 0; d < D; d++) {
        double sumC = 0.0;
        for (i = 0; i < N; i++) {
            sumC += A[i] * V[i * D + d]; // 4 FLOPS
        }
        C[n * D + d] = sumC; // 2 FLOPS
    }
}

if (!debug_mode) {
    t1 = get_time();
}

/* --------------------------
   5) Salida
   -------------------------- */
if (debug_mode) {
    printf("C =\n");
    for (n = 0; n < N; n++) {
        for (d = 0; d < D; d++) {
            printf("%6.1f ", C[n * D + d]);
        }
```

```
            printf("\n");
        }
    } else {
        printf("Tiempo: %f s\n", t1 - t0);
    }

    /* --------------------------
       6) Liberar memoria
       -------------------------- */
    free(X);  free(C);
    free(WK); free(WV); free(WQ);
    free(bK); free(bV); free(bQ);
    free(K);  free(V);
    free(q);  free(A);

    return 0;
}
```